

Dexios: A V1 Format for Encrypting and Storing Sensitive Data

brxken128

March 10, 2026

Abstract

Dexios is an open-source file encryption tool written in Rust. Its current normal format combines a compact V1 header, authenticated streaming encryption, memory-hard password hashing, and keyslot-based master-key wrapping.

1 Introduction

Dexios is aimed at encrypting files and packed directory archives for storage or transfer. The project separates the user-facing CLI, the workflow layer, and the low-level cryptographic primitives into different crates so that the format-critical code remains explicit and auditable.

2 Modes of Operation

This section describes the current Dexios workflows, including encryption, decryption, packing, unpacking, deletion semantics, and hashing.

2.1 Encryption

New encryption uses the LE31 stream object provided by the `aead` crate. Payloads are processed in 1 MiB blocks.

For new files, Dexios:

- generates a random 32-byte master key,
- generates a 20-byte payload nonce,
- generates salts and 24-byte wrapping nonces for keyslots,
- hashes the user-provided key material with the selected KDF,
- wraps the master key inside one or more keyslots,
- serializes the V1 header,
- encrypts the payload in stream mode.

The current CLI writes V1 headers. The first 32 bytes of the V1 header are authenticated as AAD with each encrypted block.

2.1.1 File-size Edge Cases

If the plaintext size is exactly divisible by the 1 MiB block size, Dexios still emits a final authenticated last block. This avoids ambiguity at the end of the stream.

2.1.2 Detached Mode

Dexios can store the header separately from the encrypted payload. In this mode, the header is written to another file and the ciphertext itself starts immediately with encrypted data.

Detached mode is useful when the user wants to keep the header under separate control, but header restoration is generally unsupported unless the target file already contains enough zero bytes at the beginning to hold the header.

2.2 Decryption

Decryption reads and deserializes the header first.

For V1 files, Dexios iterates through the available keyslots and attempts to derive a wrapping key from the supplied key material and the keyslot salt. If one of the derived keys successfully decrypts the wrapped master key, that master key is used to decrypt the payload stream.

Legacy inspection and compatibility helpers still exist in the codebase, but they are no longer the normal product surface.

2.3 Packing

Packing is an extension of encryption.

The current implementation accepts one or more input directories, derives unique archive roots for them, creates a temporary zip archive, and then encrypts that archive with the normal Dexios file workflow. Recursive traversal is the default behavior; the `--recursive` flag is retained only as a compatibility alias.

ZSTD compression is available but must be explicitly enabled.

The packed payload is still just an encrypted zip archive. Users may decrypt it normally and handle the zip archive externally if they want.

The temporary plaintext archive created during pack is handled as an ordinary temporary file and dropped after the workflow finishes.

2.4 Unpacking

Unpacking is an extension of decryption.

The source file is first decrypted into a temporary zip archive. Dexios then normalizes archive paths, rejects traversal attempts, rejects duplicate output paths after normalization, rejects unsafe output symlink escapes, and extracts the archive into the chosen directory.

The temporary plaintext archive created during unpack is handled as an ordinary temporary file and dropped after extraction.

2.5 Deletion Semantics

Dexios no longer exposes secure-erase behavior as a product feature.

Instead, selected workflows can delete source inputs after successful completion:

- `encrypt --delete-input`
- `decrypt --delete-input`
- `unpack --delete-input`
- `pack --delete-source`

If an operation fails, its source inputs are retained.

2.6 Hashing

BLAKE3 is the only hashing algorithm exposed by Dexios' checksum mode.

This mode is useful for manual or out-of-band verification of files, but it is not the primary integrity mechanism of the Dexios format. Ciphertext and header integrity are already enforced by authenticated encryption and AAD validation.

3 Key Operations

Dexios can modify V1 keyslots because the payload AAD covers only the first 32 bytes of the V1 header, not the keyslot area.

3.1 Key Changing

Changing a key decrypts the wrapped master key with the old credential, derives a new wrapping key from the replacement credential, and rewrites the first matching keyslot with a new salt and nonce.

3.2 Key Addition

Adding a key decrypts the wrapped master key and appends a new V1 keyslot containing a new wrapped copy of that same master key.

The current V1 format allows up to 4 keyslots.

3.3 Key Deletion

Key deletion identifies the first keyslot that can be opened with the supplied old credential and removes that keyslot from the header.

All current V1 key-manipulation operations act on the first matching keyslot rather than all matching keyslots.

4 Cryptography

4.1 Password Hashing

Dexios uses memory-hard password hashing to derive 32-byte wrapping keys from user-supplied key material.

All current KDFs use a 16-byte salt.

4.1.1 Hashing Algorithms

The current code exposes:

- Argon2id
- BLAKE3-Balloon

For new files:

- the default is Blake3Balloon
- --argon selects Argon2id

4.1.2 Performance

The KDF parameters are intentionally expensive compared to a general-purpose password hash, but still chosen to remain practical on normal user hardware. In most real workflows, storage I/O dominates total runtime once the key has been derived.

4.2 Encryption Algorithms

The current CLI supports one suite for new output:

- XChaCha20-Poly1305
- LE31 stream mode

4.3 Threat Model

Dexios is designed to protect file contents and format integrity at rest and in transit. It does not attempt to defend against a fully compromised host that can observe the running process directly.

4.4 Key Inputs

Dexios currently accepts key material from:

- an interactively entered passphrase,
- a keyfile,
- a generated passphrase,
- or the DEXIOS_KEY environment variable.

The current CLI resolves these sources in this order:

1. explicit keyfile,
2. explicit generated passphrase request,
3. environment variable,
4. interactive prompt.

4.4.1 Passphrase Autogeneration

The current implementation generates n random words from a bundled wordlist and joins them with -.

The default is 7 words:

word-word-word-word-word-word-word

4.5 Cryptographic Hygiene

Dexios uses the zeroize crate and a Protected wrapper to reduce accidental exposure of secrets in memory.

4.5.1 Protected Wrapper

The Protected wrapper provides explicit access through `.expose()`, redacts debug output, forbids implicit copying, and zeroizes the wrapped value on drop.

4.6 RNGs

Random values are generated through the current `rand` API, which draws from operating system entropy sources.

5 Headers

The current V1 header length is 416 bytes.

5.1 Header Structure

The V1 layout is:

{Magic | Version | KeyslotCount | Reserved | DataNonce | Reserved | KeyslotArea}

The 4-byte magic is DXIO.

The 2-byte version field is 0x0001.

Only the first 32 bytes of the V1 header are authenticated as AAD.

5.2 Keyslots

The V1 keyslot area contains space for 4 keyslots at 96 bytes each.

Each keyslot contains:

{Identifier | WrappedMasterKey | Nonce | Salt | Reserved}

The wrapped master key is 48 bytes long.

The identifier encodes the KDF family used for that keyslot. Current identifiers are:

- [0xDF, 0x01] for Blake3Balloon
- [0xDF, 0x02] for Argon2id

5.3 AAD

For V1, the first 32 bytes of the header are authenticated as AAD. This protects the magic, version, keyslot count, payload nonce, and static reserved bytes while still allowing keyslot updates without re-encrypting the payload.

5.4 Header Operations

Dexios supports:

- stripping headers by zeroing the serialized header region,
- dumping valid headers,
- restoring headers only when the target starts with enough zero bytes.

Files created in detached-header mode generally do not reserve enough zero space for restoration.

6 Conclusion

Dexios' current design favors a compact V1 header, explicit keyslot handling, modern authenticated encryption, and a relatively small implementation surface. The project is best understood as a practical at-rest file-encryption format rather than a general-purpose secure messaging or secret-management system.

References

1. <https://csrc.nist.gov/projects/block-cipher-techniques/bcm>
2. <https://research.nccgroup.com/2020/02/26/public-report-rustcrypto-aes-gcm-and-chacha20poly1305-implementation-review/>
3. <https://www.eff.org/document/passphrase-wordlists>
4. <https://eprint.iacr.org/2016/027.pdf>